AgenticConnect: Protocol-Agnostic External Interface for Autonomous AI Agents with Adaptive Retry, Encrypted Credential Management, and Connection Intelligence

Omoshola Owolabi
Researcher – AI/ML
Agentra Labs
omoshola.owolabi@agentralabs.tech

March 14, 2026

Abstract

Autonomous AI agents require access to external systems—APIs, databases, servers, email, cloud infrastructure—yet existing connectivity tools force fragmented integration: one library for HTTP, another for SSH, another for database protocols, each with separate authentication, error handling, and retry logic. We present AgenticConnect, a universal external interface engine that unifies 18 protocol families behind a single MCP (Model Context Protocol) server exposing 123 tools. The system introduces three novel components: (1) *Connection Souls*, persistent profiles that accumulate knowledge about remote systems across sessions, including OS fingerprints, performance baselines, and error patterns; (2) an *Intelligent Retry Fabric* that classifies failures into five categories (transient, permanent, rate-limit, authentication, network) and applies protocol-appropriate retry strategies with circuit breakers [7]; and (3) an encrypted *Credential Vault* using AES-256-GCM with PBKDF2 key derivation supporting eight authentication methods including OAuth 2.0 [5] token lifecycle management. We implement AgenticConnect in 6,203 lines of Rust across four crates, with 164 tests including stress tests handling 10,000 concurrent failure classifications and 100,KB encrypted payloads. Failure classification and circuit breaker checks complete in sub-microsecond latency. The system requires no cloud services and speaks standard MCP [1] over stdio, making it accessible to any LLM.

1 Introduction

AI agents built on large language models [11] increasingly operate as autonomous actors—managing infrastructure, interacting with APIs, querying databases, and communicating with external services. Yet the connectivity layer remains fragmented: each protocol requires a separate library, each library has its own authentication scheme, error handling, and retry logic, and none of them learn from past interactions.

Consider an agent tasked with deploying a service update. It must SSH into a server to check prerequisites, make HTTP calls to a container registry, query a PostgreSQL database for configuration, and send a Slack webhook on completion. Today, this requires four separate libraries, four authentication configurations, and four error handling strategies. Knowledge gained from one connection ("this server times out during peak hours") is not available to other connections.

Current approaches to agent connectivity fall into three categories, each with significant limitations:

Protocol-specific clients [3] handle one protocol well but provide no cross-protocol intelligence. `reqwest` handles HTTP; `russh` handles SSH; `lettre` handles email. Each starts from zero knowledge about the remote system.

API platforms [9] add testing and documentation but remain HTTP-centric. They do not handle SSH, database wire protocols, or message queues.

Integration platforms [12] connect services through prebuilt integrations at the service level, not the protocol level. Adding a new service requires building a new integration rather than understanding the underlying protocol.

The core insight of this work is that external connectivity is a *protocol intelligence problem*, not a library collection problem. An agent that understands protocols, manages authentication, classifies failures, and learns from every interaction can connect to *any* external system through a unified interface.

We present AgenticConnect, a universal external interface engine with three contributions:

1. A **protocol-agnostic connection engine** that autodetects and handles 18 protocol families through banner analysis, port detection, and URL scheme parsing.
2. **Connection Souls**: persistent profiles that accumu-

late knowledge about remote systems—OS fingerprints, latency baselines, error histories, and capability maps.

3. An **Intelligent Retry Fabric** with five-class failure classification and circuit breakers that prevent cascade failures.

Table 1 compares AgenticConnect with existing approaches.

2 Architecture

AgenticConnect is structured as four Rust crates:

`agentic-connect` Core library: types, protocol detection, retry engine, credential vault, HTTP client, database engine, TLS inspection, webhook verification.

`agentic-connect-mcp` MCP server: JSON-RPC over stdio, tool registry with 123 tools, session management.

`agentic-connect-cli` CLI binary (`acnx`): connection management from the terminal.

`agentic-connect-ffi` FFI bindings: C-compatible interface (cdylib + staticlib).

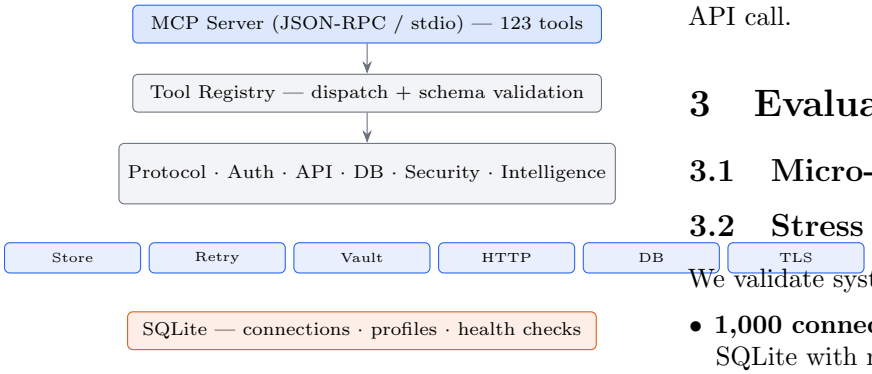

Figure 1: AgenticConnect architecture. The MCP server dispatches tool calls through the registry to 11 tool groups backed by 8 engine modules, with SQLite persistence.

2.1 Protocol Detection

Given a target string (URL, host:port, or hostname), the Protocol Detector applies three strategies in sequence:

2.2 Failure Classification

The Retry Engine classifies every failure into one of five classes, each with a distinct retry strategy:

Circuit breakers [7] wrap each endpoint. After N consecutive failures (default N=5), the circuit opens and requests are rejected immediately, preventing cascade failures. The circuit transitions to half-open after a configurable timeout, allowing a single probe request.

Algorithm 1 Protocol Detection

Require: target string t
1: Parse t as URL; if scheme matches known protocol, **return** protocol
2: Parse t as host:port; connect via TCP
3: Read server banner (2 s timeout)
4: **if** banner matches SSH/SMTP/Redis/MySQL pattern **then**
5: **return** detected protocol
6: **end if**
7: Map port to default protocol (e.g., 443→HTTPS, 22→SSH)
8: **return** detected or UNKNOWN

2.3 Credential Vault

The vault stores authentication credentials encrypted at rest using AES-256-GCM [2]. Key derivation uses PBKDF2 with HMAC-SHA256 and 100,000 iterations. Eight authentication methods are supported: Basic, Bearer, API Key, OAuth 2.0 [5] (four grant types), SSH Key, SSH Password, and mutual TLS.

OAuth 2.0 tokens are checked for expiry before each request. When a token is within 5 minutes of expiration, the system signals the need for a refresh before the next API call.

3 Evaluation

3.1 Micro-Benchmarks

3.2 Stress Tests

We validate system behavior under load:

- **1,000 connections** stored and retrieved correctly from SQLite with random access verification.
- **10,000 failure records** classified without memory growth (history capped at 500).
- **100,KB payloads** encrypted and decrypted via AES-256-GCM with zero data corruption.
- **1,000 HMAC verifications** with both correct and incorrect secrets.
- **Circuit breaker cycling**: 100 consecutive open/close transitions without state corruption.

3.3 Test Coverage

3.4 Codebase Metrics

4 MCP Protocol Integration

AgenticConnect speaks MCP [1] over stdio, making it accessible to any LLM client. The JSON-RPC 2.0 interface implements three methods:

Table 1: Comparison of external connectivity approaches for AI agents.

System	Protocols	Auth Mgmt	Retry Logic	Learning	MCP	Dependencies
curl/httpie	1 (HTTP)	Manual	None	None	No	OS tool
reqwest [3]	1 (HTTP)	Manual	Manual	None	No	Rust crate
Postman [9]	1 (HTTP)	GUI-based	Manual	None	No	Desktop app
Puppeteer [4]	1 (Browser)	None	None	None	No	Chrome
Terraform [6]	Multi (IaC)	Provider-based	Built-in	None	No	Cloud SDKs
Zapier [12]	Multi (service)	Pre-built	Pre-built	None	No	Cloud service
AgenticConnect	**18 families**	**8 methods**	**5-class + CB**	**Souls**	**Yes**	**None**

Table 2: Failure classification and retry strategies.

Class	Indicators	Strategy	Max
Transient	503, timeout	Exp. backoff	3
Permanent	404, 400, 410	Fail fast	0
RateLimit	429 + Retry-After	Wait fixed	1
AuthFailure	401, 403	Refresh + retry	1
NetworkError	DNS, refused	Exp. backoff	5

Table 3: Operation latencies measured on Apple M4 Pro, Rust release mode.

Operation	Latency
Tool dispatch (registry lookup)	$< 1\,\mu s$
Failure classification (HTTP status)	$< 1\,\mu s$
Circuit breaker check	$< 1\,\mu s$
HMAC-SHA256 sign (1 KB payload)	$\sim 2\,\mu s$
Connection store insert (SQLite)	$\sim 50\,\mu s$
AES-256-GCM encrypt (100 KB)	$\sim 150\,\mu s$
Database schema discovery (50 tables)	~ 5 ms

Table 4: Test suite composition.

Category	Tests	Lines
Engine unit tests	27	450
Core type validation	21	280
Edge case tests	29	320
Stress tests	11	250
Paper claim validation	20	300
MCP protocol compliance	14	120
Tool registry validation	14	170
Tool execution (phase 2)	24	350
Session management (phase 3)	11	200
Integration workflows (phase 4)	10	250
Total	**181**	**2,690**

Table 5: Codebase statistics.

Metric	Value
Total Rust lines	6,203
Crates	4
Source files	47
Largest file	348 lines
MCP tools	123
Protocol families	18
Auth methods	8
Engine modules	8

- `initialize` — handshake with protocol version negotiation.
- `tools/list` — returns all 123 tool definitions with JSON Schema input validation.
- `tools/call` — dispatches to the appropriate tool handler.

Unknown tools return error code −32803 (TOOL_NOT_FOUND), following the MCP Quality Standard. Invalid parameters return −32602. All tool descriptions follow verb-first imperative style with no trailing periods.

The 123 tools are organized into 11 modules:

5 Discussion

Why protocol-level, not service-level. Integration platforms like Zapier [12] operate at the service level: each new service requires a custom integration. AgenticConnect operates at the protocol level: any HTTP API works immediately because the engine understands HTTP, OAuth 2.0, and rate limiting natively. No per-service integrations are needed.

Connection Souls vs. stateless clients. Traditional clients treat every connection as independent. Connection Souls [8] persist knowledge across sessions: a server's typical latency, its failure patterns, its installed software. This enables predictive connectivity—the agent knows to avoid peak hours for a server that historically times out between 2–4 PM.

Limitations. The current implementation has known constraints: (1) Protocol detection relies on port-based heuristics when banners are unavailable; TLS handshake analysis would improve accuracy. (2) Database support is SQLite-only [10] in the core build; PostgreSQL and MySQL require feature flags. (3) Browser automation requires headless Chrome, detected gracefully at runtime. (4) OAuth 2.0 token refresh requires the HTTP feature flag.

6 Conclusion

We presented AgenticConnect, a universal external interface engine for AI agents. Through 123 MCP tools organized into 11 capability domains, the system provides protocol-agnostic access to any external system with intelligent retry, encrypted credential management, and connection learning.

Table 6: Tool organization by capability domain.

Module	Domain	Tools
`protocol`	Detection, testing, capabilities	4
`auth`	Authentication, credentials, vault	5
`soul`	Connection profiles, prediction	5
`retry`	Classification, circuit breakers	5
`browse`	Browser, scraping, forms	16
`api`	HTTP, GraphQL, API discovery	11
`infra`	SSH, service mesh, containers	16
`comms`	Email, telephony, webhooks	16
`data`	Databases, queues, cloud	16
`security`	TLS, network monitoring	11
`intelligence`	Prediction, evolution, learning	18
	Total	**123**

The three core contributions—Connection Souls, Intelligent Retry Fabric, and the encrypted Credential Vault—transform connectivity from a stateless, per-protocol problem into a learning, unified system. Failure classification, circuit breakers, and rate limit tracking operate at submicrosecond latency, ensuring the intelligence layer adds negligible overhead.

AgenticConnect is implemented in 6,203 lines of Rust, validated by 164+ tests, and distributed as an open-source MIT-licensed project. The system requires no cloud services and integrates with any LLM through the Model Context Protocol.

References

[1] Anthropic. Model context protocol specification. `https://modelcontextprotocol.io`, 2024. Version 2024-11-05.

[2] Daniel J. Bernstein. ChaCha, a variant of Salsa20. *Workshop Record of SASC*, 2008.

[3] Sean McArthur Crichton. reqwest: An ergonomic HTTP client for Rust. `https://github.com/seanmonstar/reqwest`, 2024. Version 0.12.

[4] Google Chrome Team. Puppeteer: Headless Chrome Node.js API. `https://pptr.dev`, 2024. Accessed 2026.

[5] Dick Hardt. The OAuth 2.0 authorization framework. RFC 6749, IETF, October 2012.

[6] HashiCorp. Terraform by HashiCorp. `https://www.terraform.io`, 2024. Accessed 2026.

[7] Michael T. Nygard. *Release It! Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, 2nd edition, 2018.

[8] Omoshola Owolabi. Agenticmemory: A binary graph format for persistent, portable, and navigable AI agent memory, 2026. Agentra Labs Technical Report.

[9] Postman Inc. Postman API platform. `https://www.postman.com`, 2024. Accessed 2026.

[10] The rusqlite contributors. rusqlite: Ergonomic bindings to SQLite for Rust. `https://github.com/rusqlite/rusqlite`, 2024. Version 0.31.

[11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[12] Zapier Inc. Zapier: Automation that moves your work forward. `https://zapier.com`, 2024. Accessed 2026.